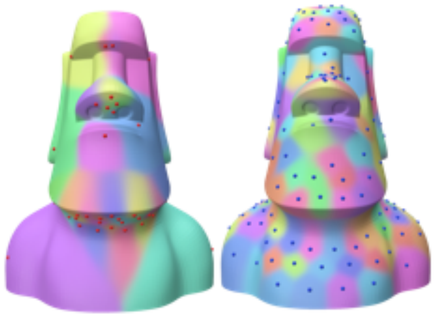
Computer Graphics

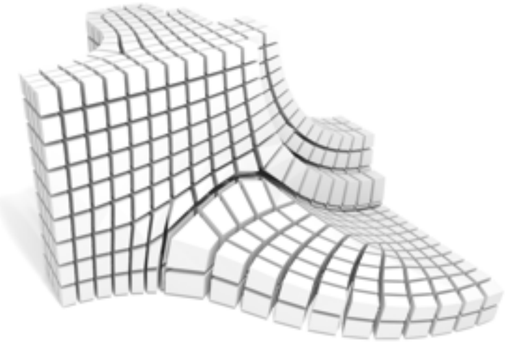
Geometric Algorithms

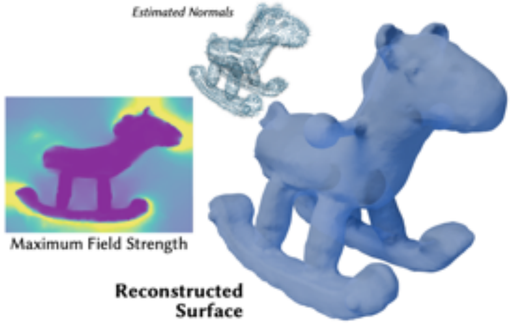
CAS CS 132

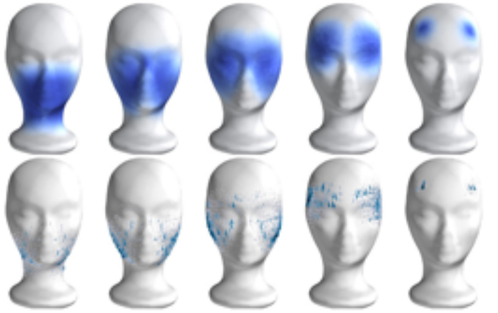
Geometry Processing

- Applying ideas from **differential geometry** and **topology** to **computer graphics** and **machine learning**





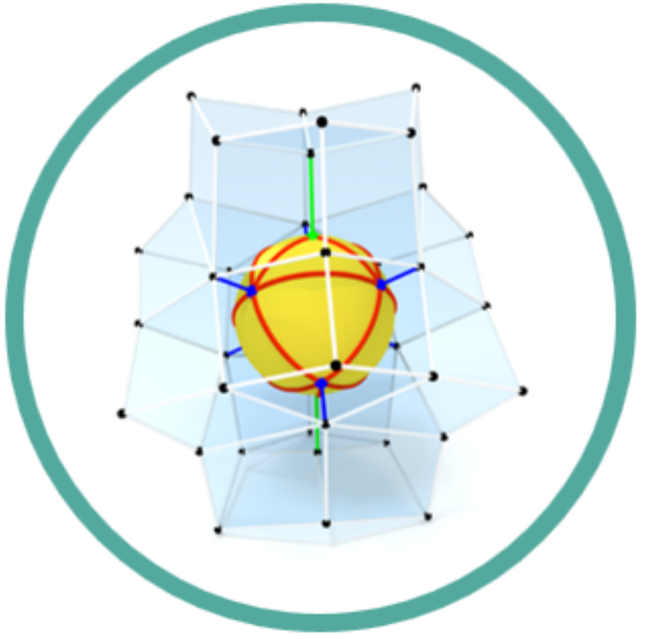




CS 680
Intro to Computer Graphics

CS 581
Computational Fabrication

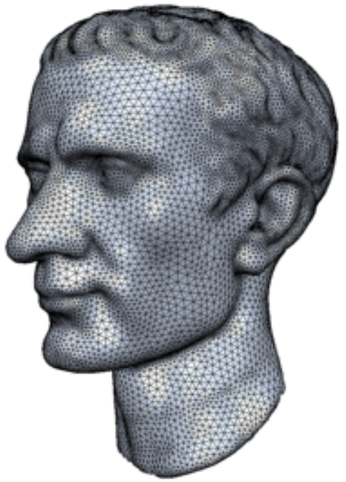
CS 591
Geometry Processing



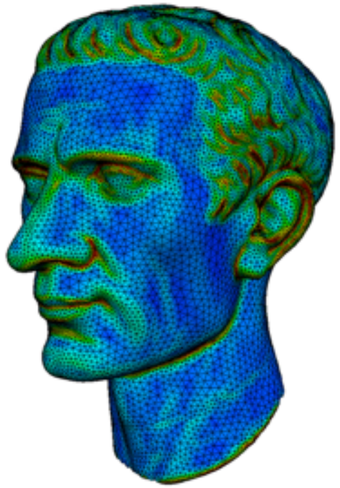
Graduate Courses

Geometry Processing

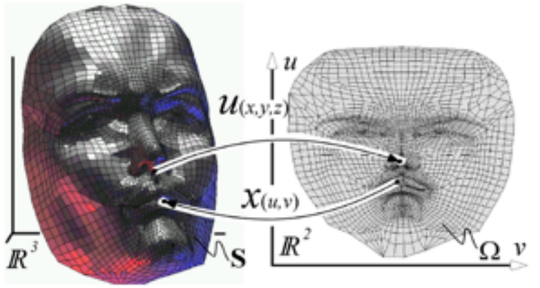
Numerical linear algebra
+
Optimization
+
Discrete differential geometry



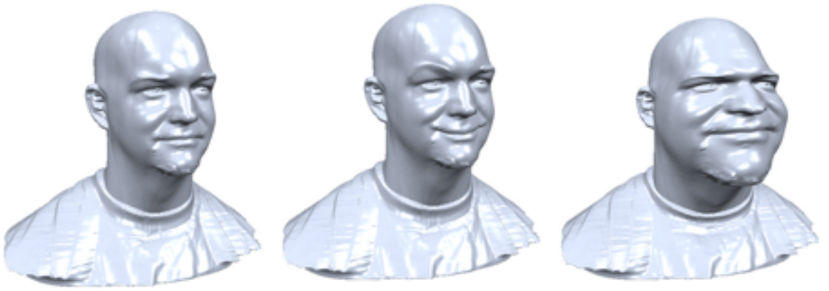
adaptive meshing



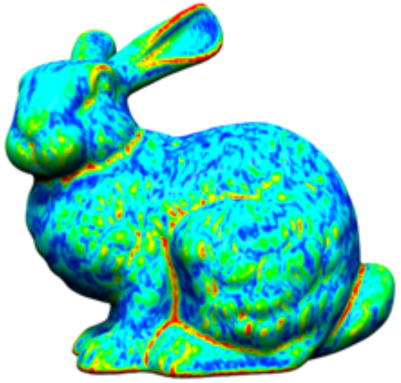
curvature visualization



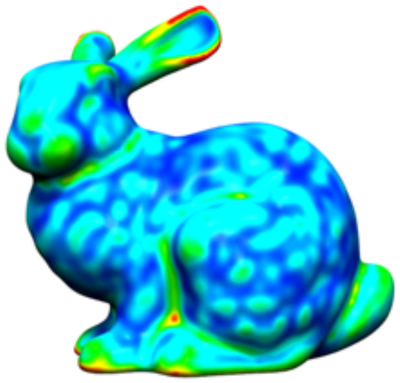
parameterization



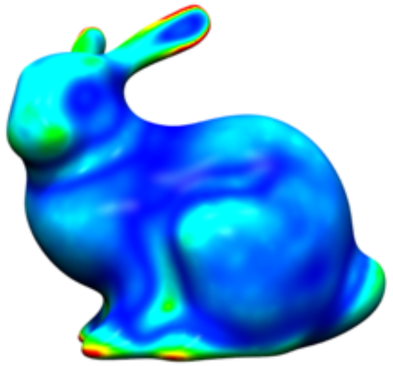
deformation



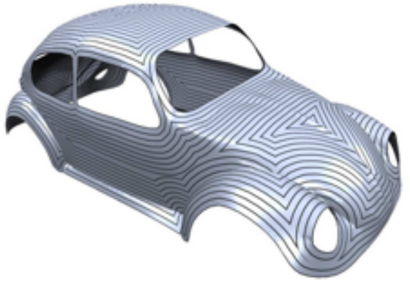
0 iterations



10 iterations



100 iterations



geodesic distances

computational fabrication: what is it?

traditional computer graphics
simulate reality

computational fabrication
digital to physical

Practice Problem

(LAA 4.9.3) On any given day a student is healthy or ill. Of the students healthy today, 5% will be ill tomorrow, and 55% of ill students will remain ill tomorrow

Write down the stochastic matrix for this situation

Objectives

Finish up our discussion of **Markov chains**

Look at linear algebraic methods in **graphics**

Keywords

elementary matrices

LU factorization

wireframe objects

homogeneous coordinates

translation

perspective projections

Recap

Recap: Linear Dynamical Systems

Def. A **(discrete time) linear dynamical system** is a described a $n \times n$ matrix A. It's **evolution function** is the matrix transformation $\mathbf{x} \mapsto A\mathbf{x}$

The **possible states** of the system are vectors in $\mathbb{R}^n$

Given an **initial state vector** $\mathbf{v}_0$, we can determine the **state vector** of the system after i time steps:

$$\mathbf{v}_i = A\mathbf{v}_{i-1}$$ *(linear recurrence relation)*

Recap: Markov Chains

Def. A **Markov chain** is a linear dynamical system whose evolution function is given by a *stochastic* matrix

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix} \begin{bmatrix} \text{city}_{k-1} \\ \text{suburb}_{k-1} \end{bmatrix}$$

Recall: Steady-State Vectors

Definition. A **steady-state vector** for a stochastic matrix A is a probability vector $\mathbf{q}$ such that

$$A\mathbf{q} = \mathbf{q}$$

A steady-state vector is *not changed* by the stochastic matrix. They describe *equilibriums*

How to: Steady-State Vectors

Question. Determine if the Markov chain with stochastic matrix A has a steady-state vector. If it does, find such a vector

Solution. Solve the equation $(A - I)\mathbf{x} = \mathbf{0}$ and find a solution whose entries sum to 1 (this will be possible given a free variable)

If there is no such solution, the system does not have a steady state

Existence vs Convergence

If $(A - I)\mathbf{x} = \mathbf{0}$ has a nontrival solutions, then the system has a stable state. This **does not** mean:

- the stable state is unique
- the system *converges* to this state

Convergence

Def. For a Markov chain with stochastic matrix A, an initial state $\mathbf{v}_0$ **converges** to the state $\mathbf{v}$ if $\lim_{k\to\infty} A^k\mathbf{v}_0 = \mathbf{v}$

As we repeatedly multiply $\mathbf{v}_0$ by A, we get closer and closer to $\mathbf{v}$ (in the limit)

(Non-)Example

I is a stochastic matrix and $I\mathbf{v} = \mathbf{v}$ for all $\mathbf{v}$

This system doesn't have a **unique** steady state, and no vectors converge to the same stable state

Regular Stochastic Matrices

Definition. A stochastic matrix A is **regular** if A^k has all positive entries for *some nonnegative* k

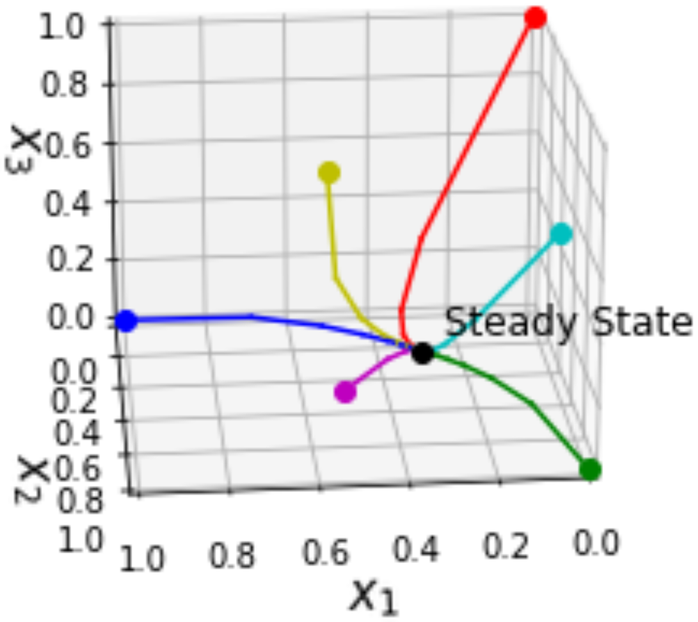
Theorem. A regular stochastic matrix P has a unique steady state, and

every probability vector converges to it

Mixing

This process of converging to a unique steady state is called "mixing"

This theorem says, after some amount of mixing, we'll be close to the stable state **no matter where we started**



Example

$$\begin{bmatrix} 0.5 & 0.4 & 0 \\ 0.5 & 0.4 & 0.5 \\ 0 & 0.2 & 0.5 \end{bmatrix}$$

State Diagrams

Def. A **state diagram** is a directed weighted graph whose adjacency matrix is stochastic.

Ex.

Naming Convention Clash

The nodes of a state diagram are often called **states**

The vectors which are dynamically updated according to a linear dynamical system are called **state vectors**

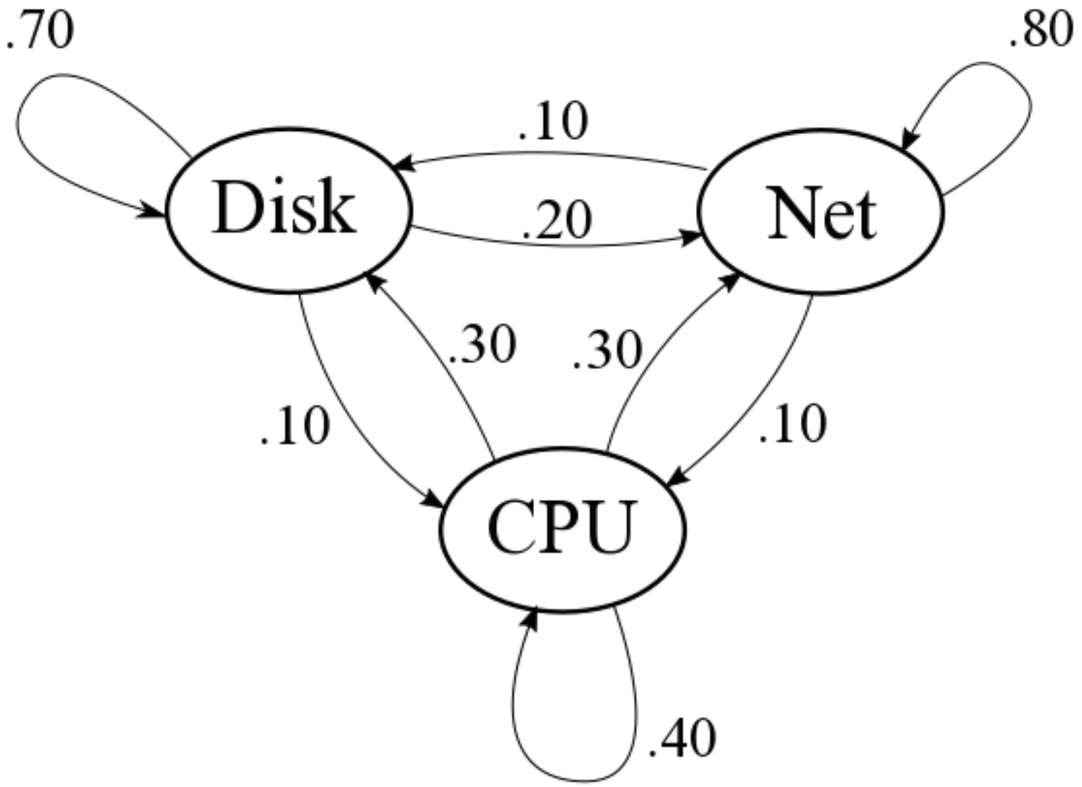
This is an unfortunate naming clash

Example: Computer System

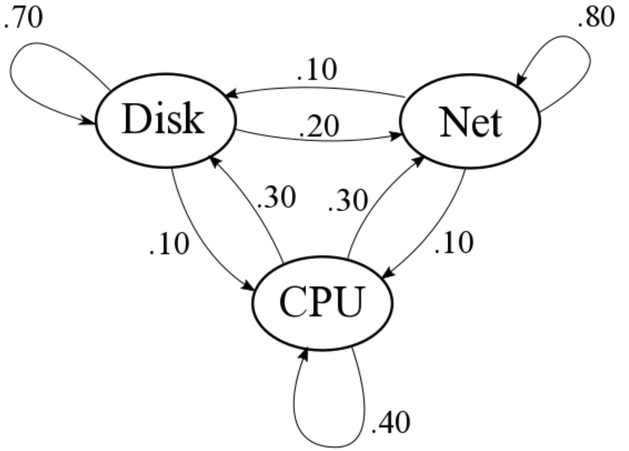
Imagine a computer system in which tasks request service from disk, network or CPU

In the long term, which device is busiest?

This is about finding a stable state



Example



moving on...

Graphics

Disclaimer

I am not an expert in this field

Motivation (or Pretty Pictures)

Graphics doesn't need much motivation

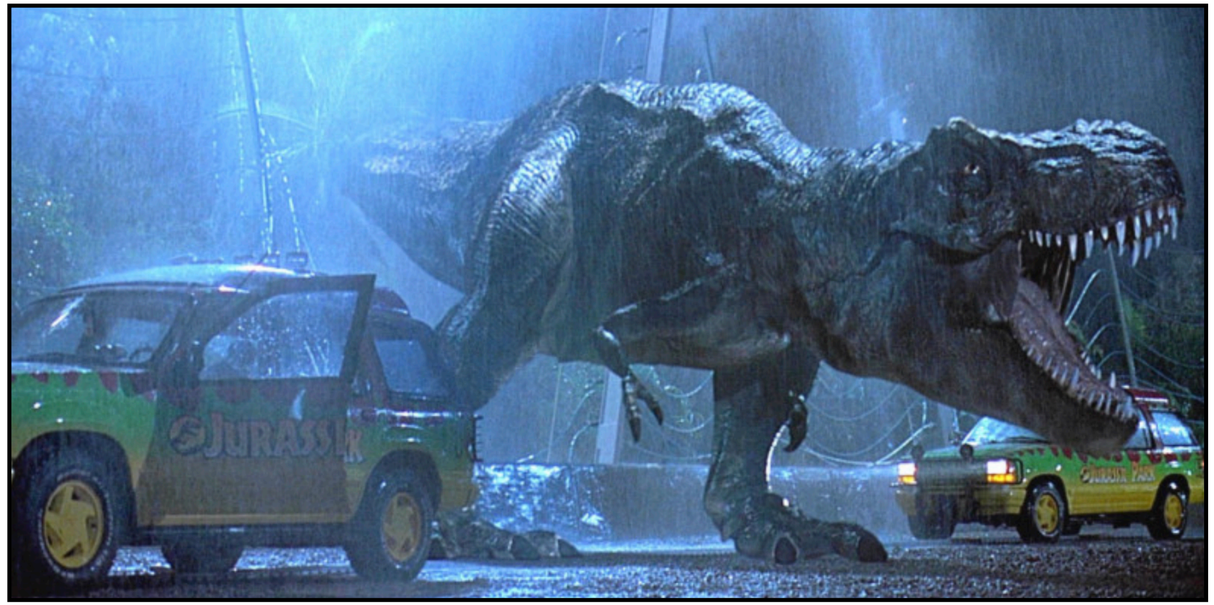
We spend so much time interacting graphics in one form or another

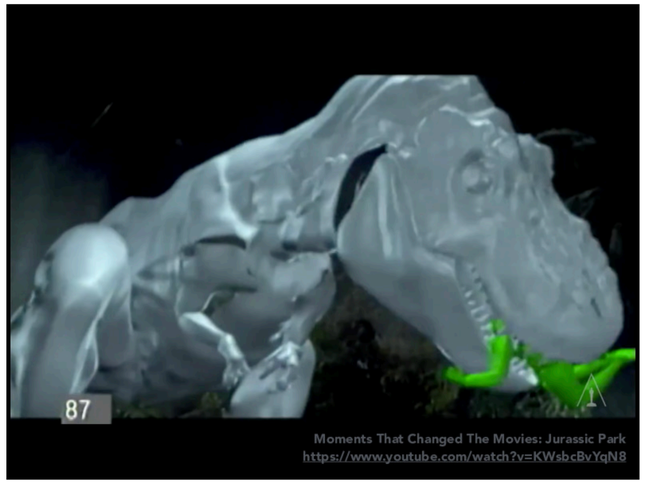
But in case you haven't thought too much about it, some examples...

source: CS184 Lecture Slides, UC Berkeley, Ng Ren

Movies

Jurassic Park (1993)





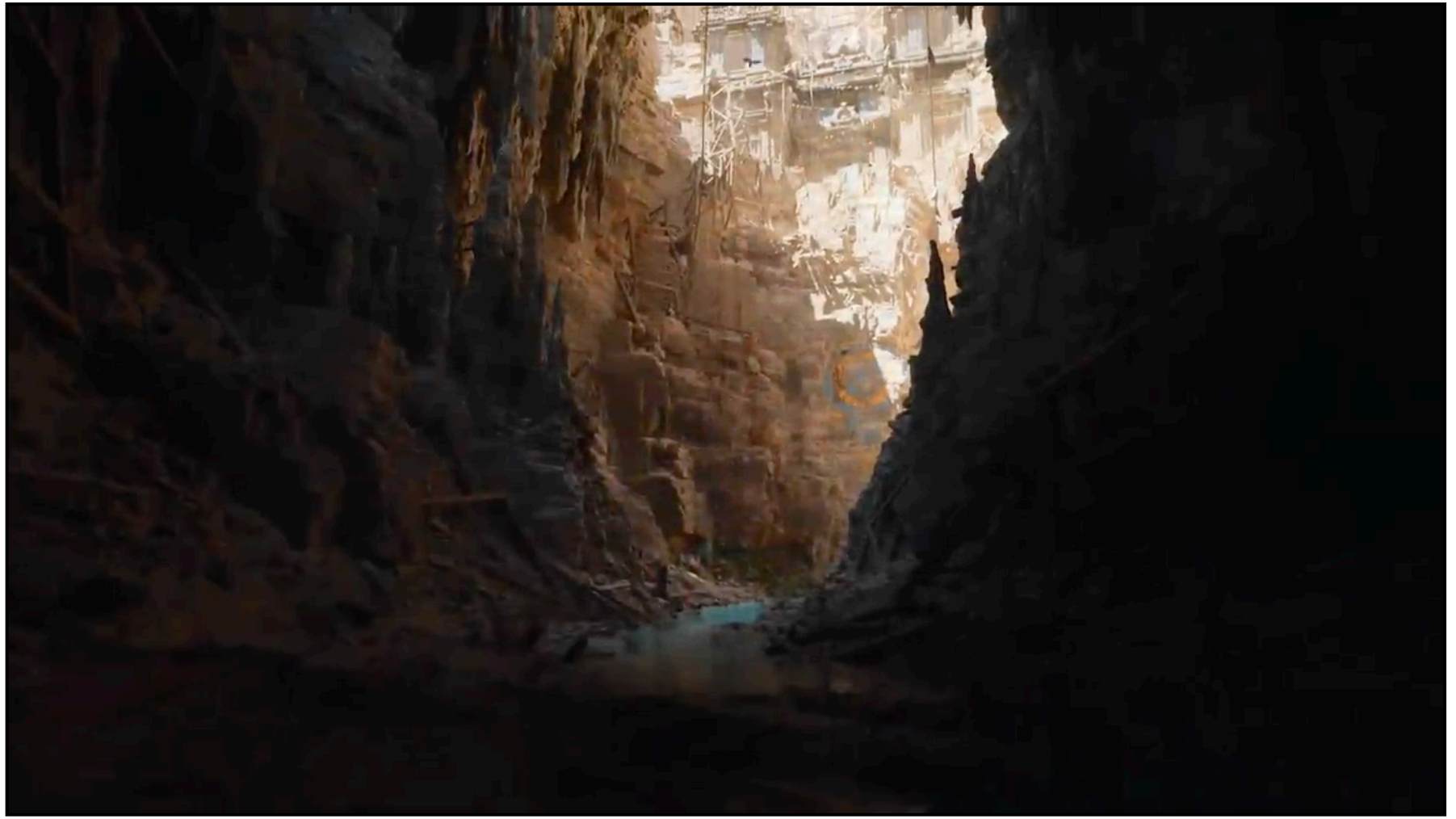
Alice in Wonderland (2010)

Motion Capture

Two Towers (2002)

Video Games

Unreal Engine 5 (2020)



Scientific Visualization

First image of a black hole (2022)

Photography

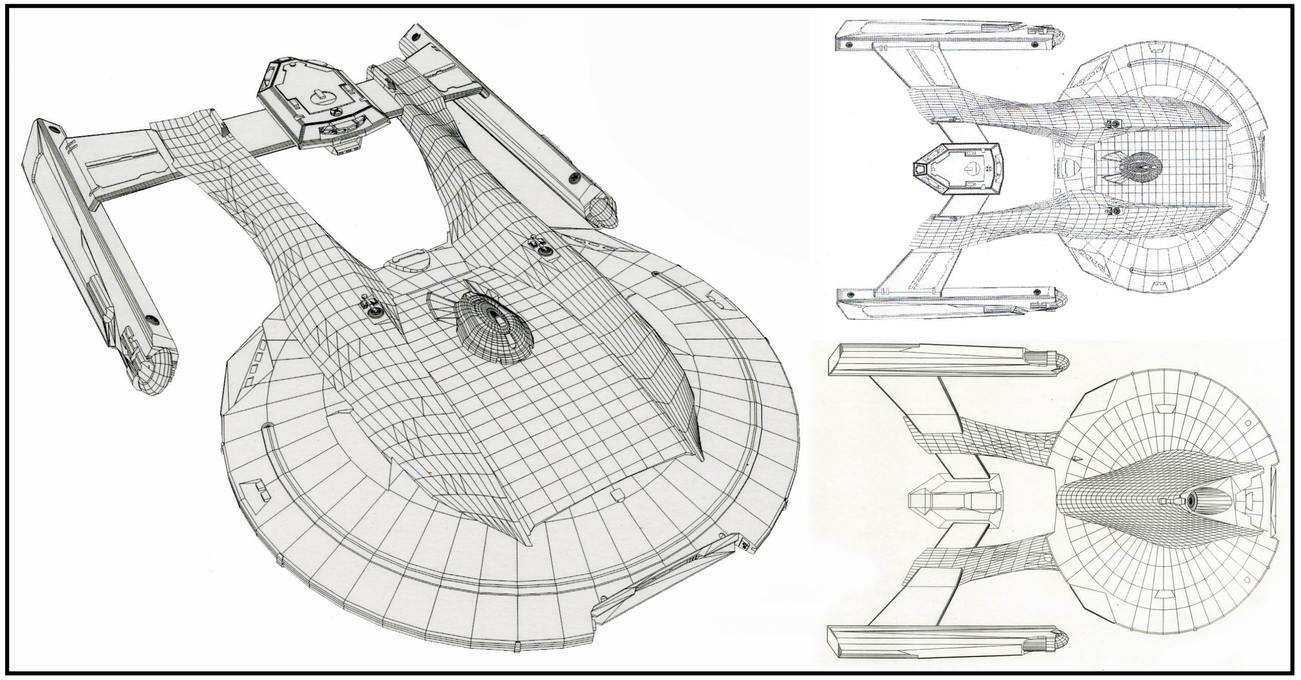
NASA | Walter Iooss | Steve McCurry
Harold Edgerton | NASA | National Geographic

Graphics & Linear Algebra

3D Graphics

There are many facets of computer graphics, but we will be focusing on one problem today:

Manipulating and Transforming 3D objects and rendering them on a screen



3D Graphics Pipeline

1. Create a 3D model of objects + scene

2. Convert the surfaces of the objects in the model into approximations called **wire frames** built out of line segments

3. Manipulate the segments ***linear*** transformations and then ***linearly*** render it in 2D (in a way that preserves perspective)

Today

Wire Frames

A **wire frame** is representation of a surface as a collection of line segments

Many useful transformations on line segments are **linear**

https://commons.wikimedia.org/wiki/File:Wireframe_Render_of_Digital_Clothing_Bathingrobe_3D_Model.jpg

Transformations

We've seen many 2D transformations:

- Reflections
- Expansion
- Shearing
- Projection

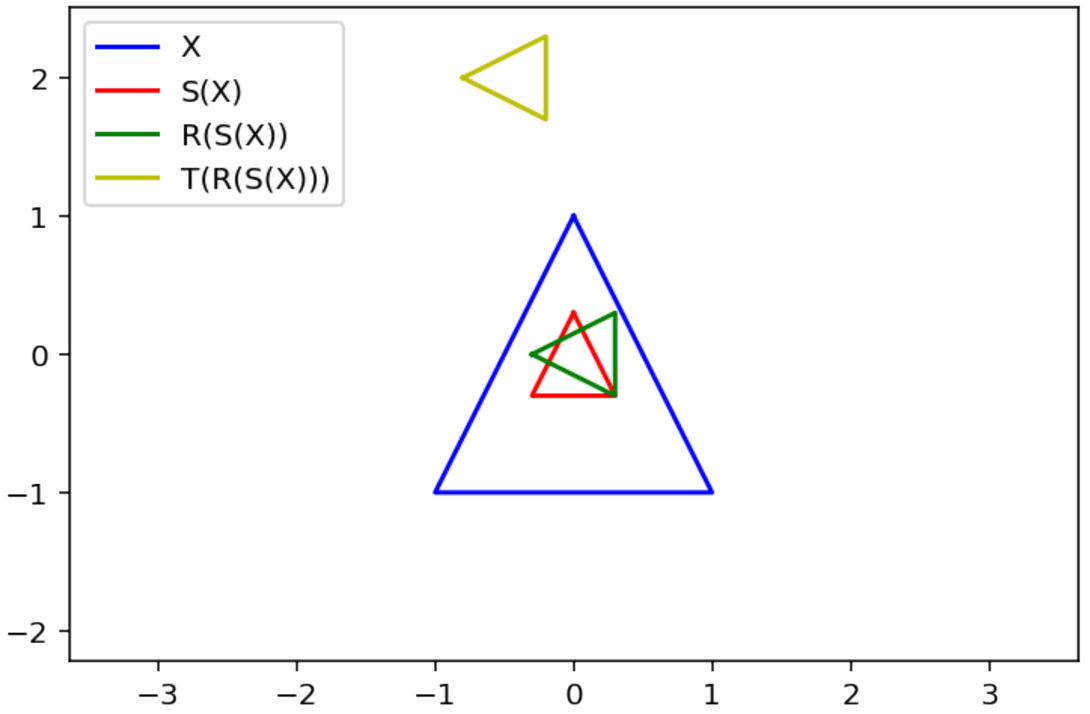
We've seen some 3D transformations:

- Rotations
- Projections

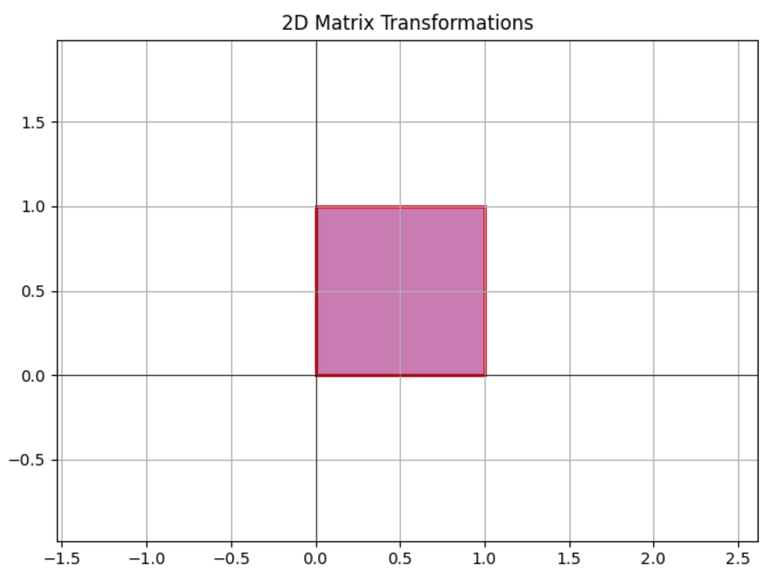
Composing Transformations

Multiplying matrices **composes** their associated transformations

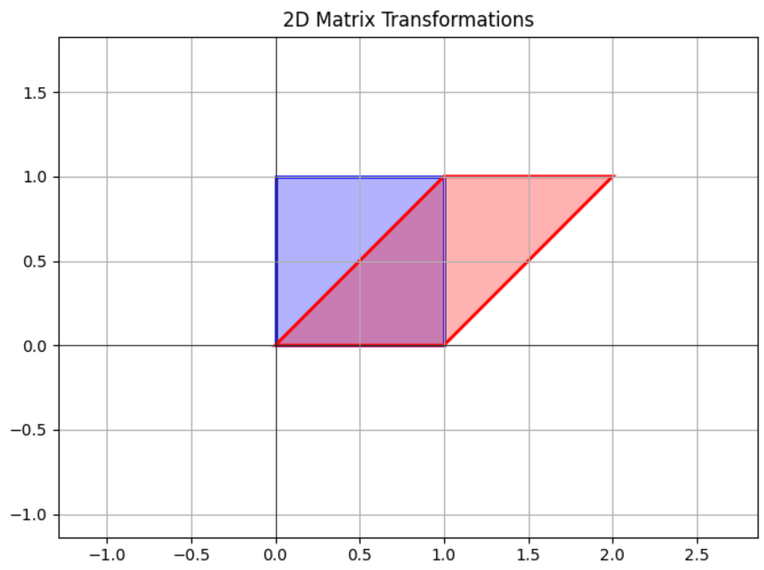
Complex graphical transformations can be *combined* into a single matrix



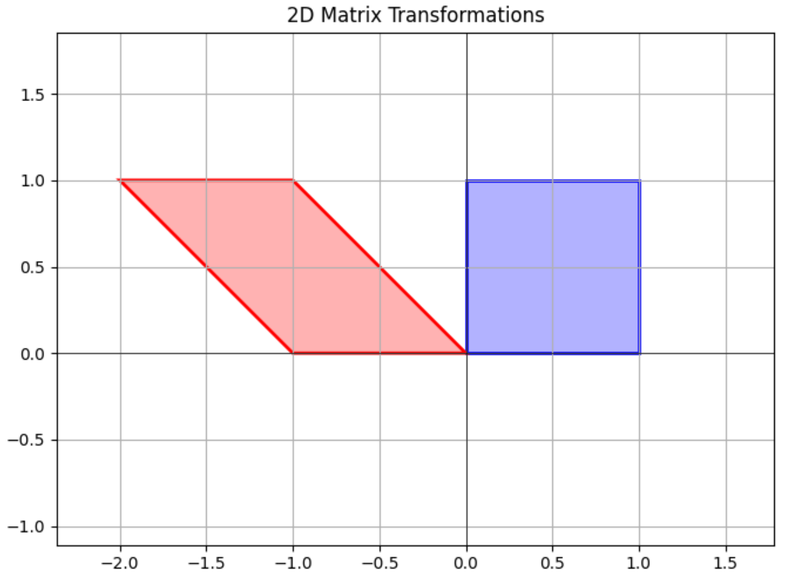
Shearing and Reflecting (Geometrically)



shear



reflect



More Transformations

What we're adding today:

- More on rotations
- translations
- perspective projections

These aren't linear, but they are incredibly important so we have to address them

3D Rotation Matrices

$$R_x^\theta = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix} \quad R_y^\theta = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \quad R_z^\theta = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

These are the matrices for counterclockwise rotation around x, y, and z axes (note the change in sign for y)

Fact. Any rotation can be done by some matrix of the form

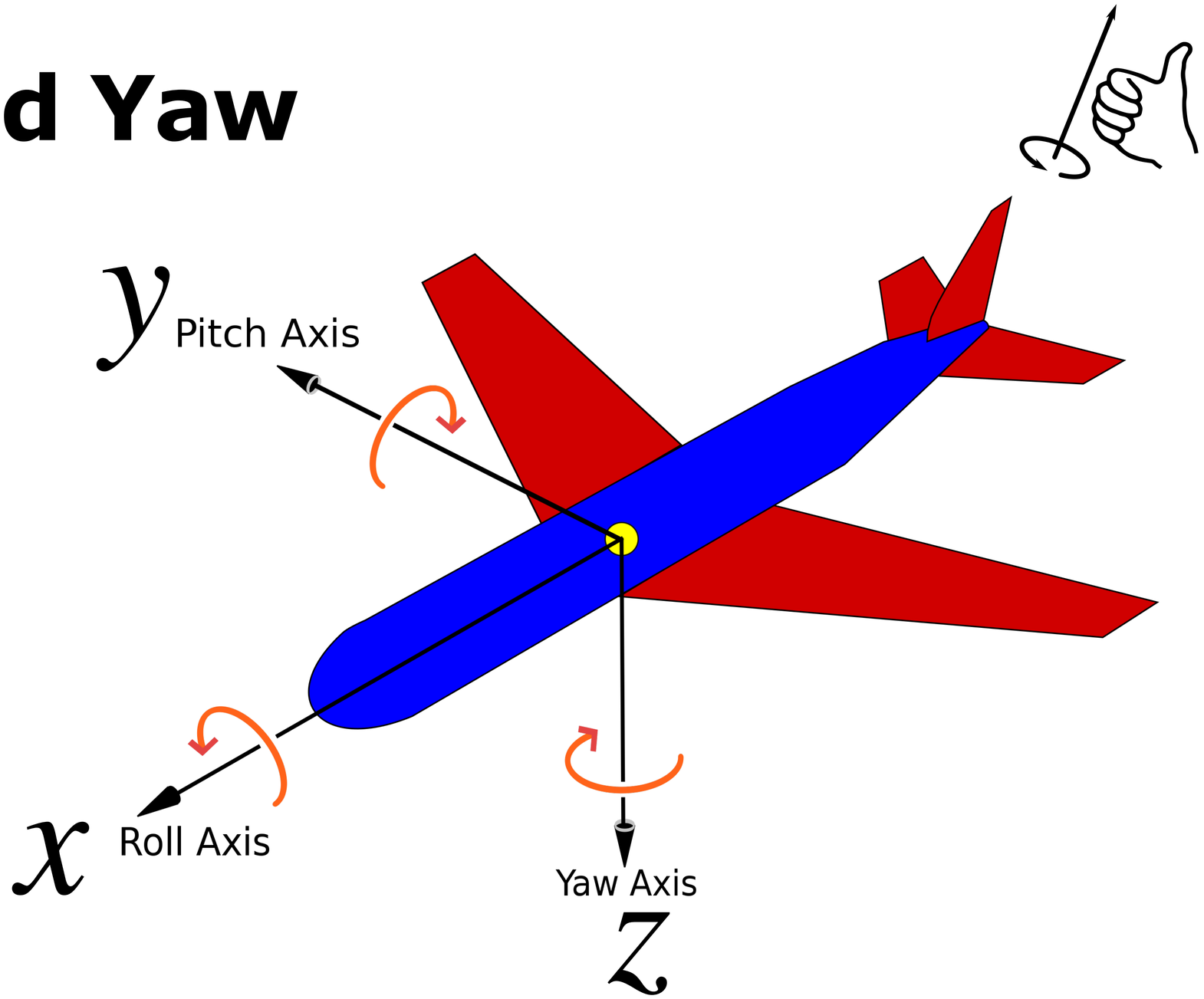
$$R_z^\theta R_y^\gamma R_x^\eta$$

Roll, Pitch and Yaw

roll
sideways tilt

pitch
up–down tilt

yaw
direction



https://commons.wikimedia.org/wiki/File:Yaw_Axis_Corrected.svg

General Rotations

$$R_z^{\theta} R_y^{\gamma} R_x^{\eta}$$

yaw pitch roll

Exactly what rotation you get is not obvious...

!!Matrix multiplication does not commute!!

Changing η doesn't necessarily rotate around the x-axis
(e.g., that axis might be tilted along the pitch axis)

demo

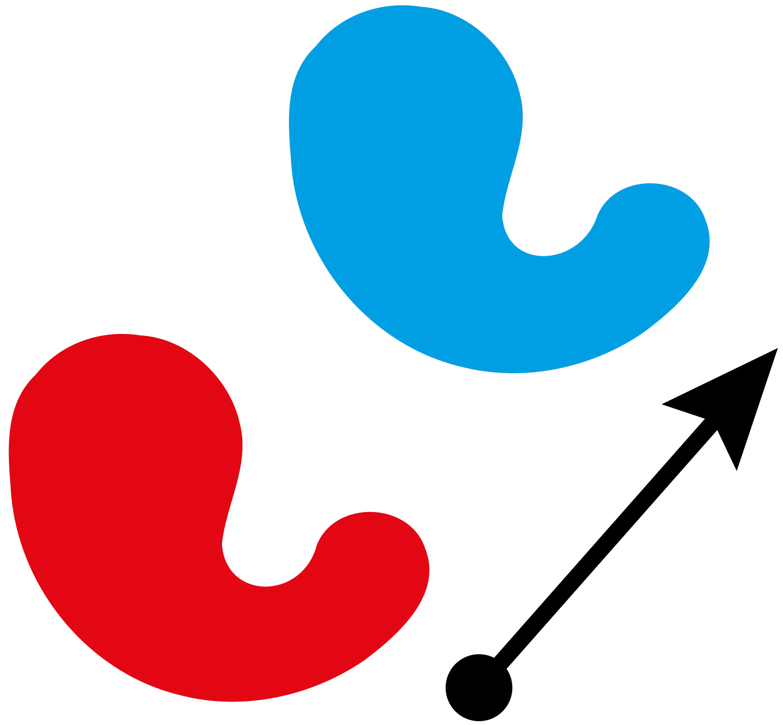
Translation

Given a vector $\mathbf{t}$ a **translation** is the transformation

For this to be interesting $\mathbf{t}$ will be nonzero

$$T(\mathbf{x}) = \mathbf{x} + \mathbf{t}$$

As we've seen, **translation is not linear**



In 2D

https://commons.wikimedia.org/wiki/File:Traslazione_OK.svg

Translation (3D)

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \mapsto \begin{bmatrix} x+a \\ y+b \\ z+c \end{bmatrix}$$

Observation. This would be linear if we had another variable

Translation (3D)

$$\begin{bmatrix} x \\ y \\ z \\ q \end{bmatrix} \mapsto \begin{bmatrix} x + aq \\ y + bq \\ z + cq \\ q \end{bmatrix}$$

Observation. This would be linear if we had another variable

Translation (3D)

$$\begin{bmatrix}1\\0\\0\\0\end{bmatrix} \mapsto \begin{bmatrix}1\\0\\0\\0\end{bmatrix} \qquad \begin{bmatrix}0\\1\\0\\0\end{bmatrix} \mapsto \begin{bmatrix}0\\1\\0\\0\end{bmatrix} \qquad \begin{bmatrix}0\\0\\1\\0\end{bmatrix} \mapsto \begin{bmatrix}0\\0\\1\\0\end{bmatrix} \qquad \begin{bmatrix}0\\0\\0\\1\end{bmatrix} \mapsto \begin{bmatrix}a\\b\\c\\1\end{bmatrix}$$

Observation. This would be linear if we had another variable

Translation (3D)

$$\begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Observation. This would be linear if we had another variable

So if we are willing to keep around an extra entry, we can do translation **linearly**

Homogeneous Coordinates

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \mapsto \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

For initializing to homogeneous coordinates, we set this to 1

Cartesian to homogeneous

The **homogeneous coordinate** for vector in $\mathbb{R}^3$ is the same except "sheared" into the 4th dimension

We use the extra entry to perform simple nonlinear transformations in a linear setting

Translation (2D)

Translation (3D)

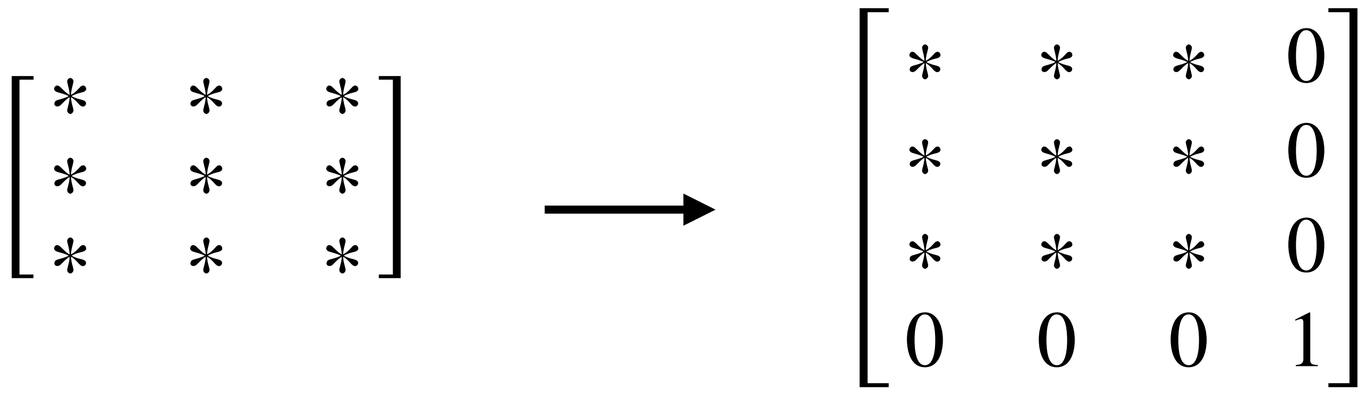
Def. The **3D translation matrix** for homogeneous coordinates which translates by $(a, b, c)^T$ is the following

$$\begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Example.

$$\begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \begin{bmatrix} x+2 \\ y+2 \\ z+2 \\ 1 \end{bmatrix}$$

Matrix Transformations for Homogeneous Coordinates



Now all our transformations need to be 4×4 matrices

It's easy make 3×3 matrices work for homogeneous coordinates

If a transformation is linear, it doesn't make use of the extra coordinate

Example: Homogeneous Rotation

Rotating counterclockwise about the x-axis in homogeneous coordinates is given by

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Perspective Projections

Vanishing Points

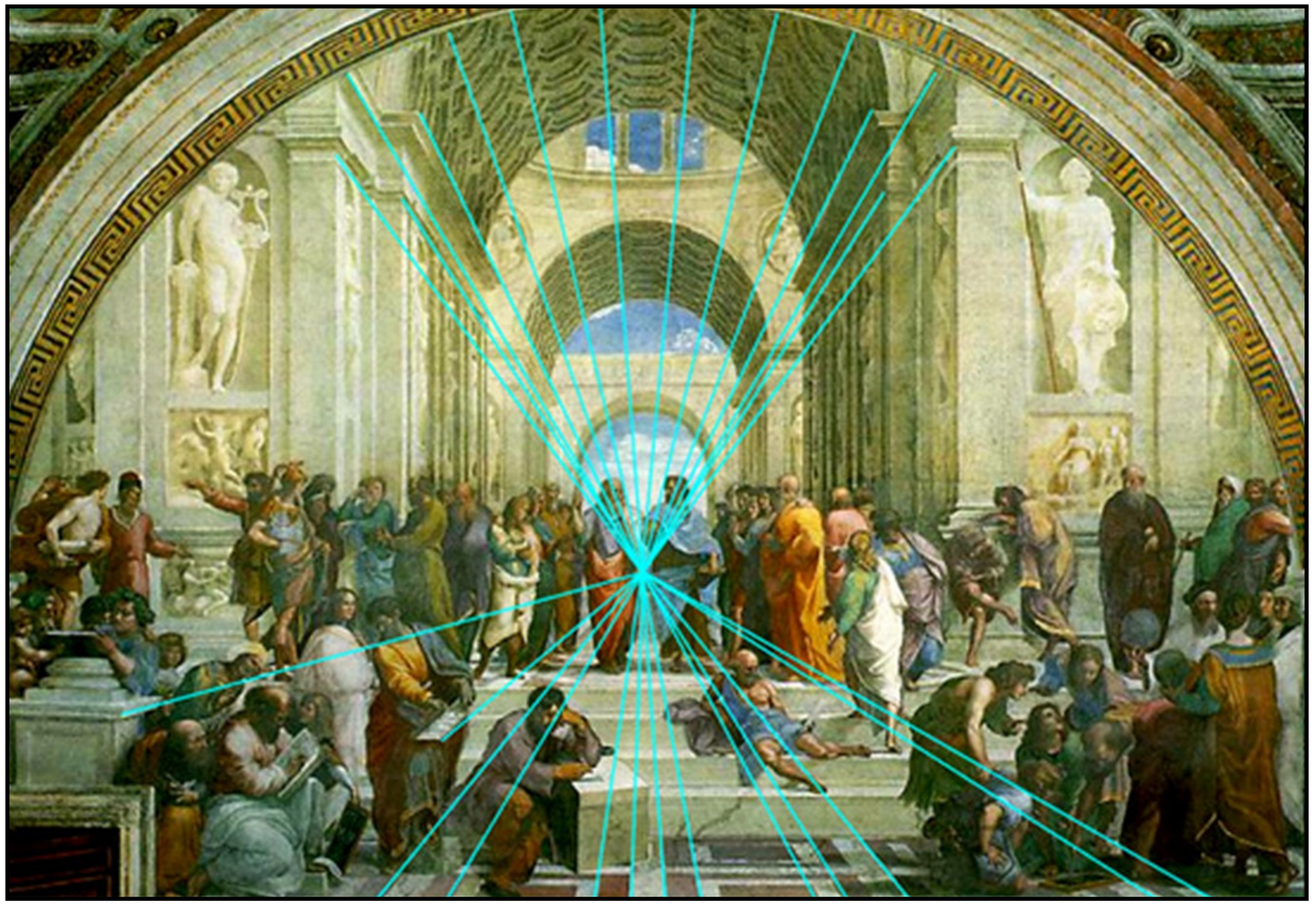
Parallel lines in space don't necessarily look parallel at a distance, they angle towards a point in the distance

This is a side effect of **perspective projection**

https://commons.wikimedia.org/wiki/File:Railroad-Tracks-Perspective.jpg

Vanishing Point

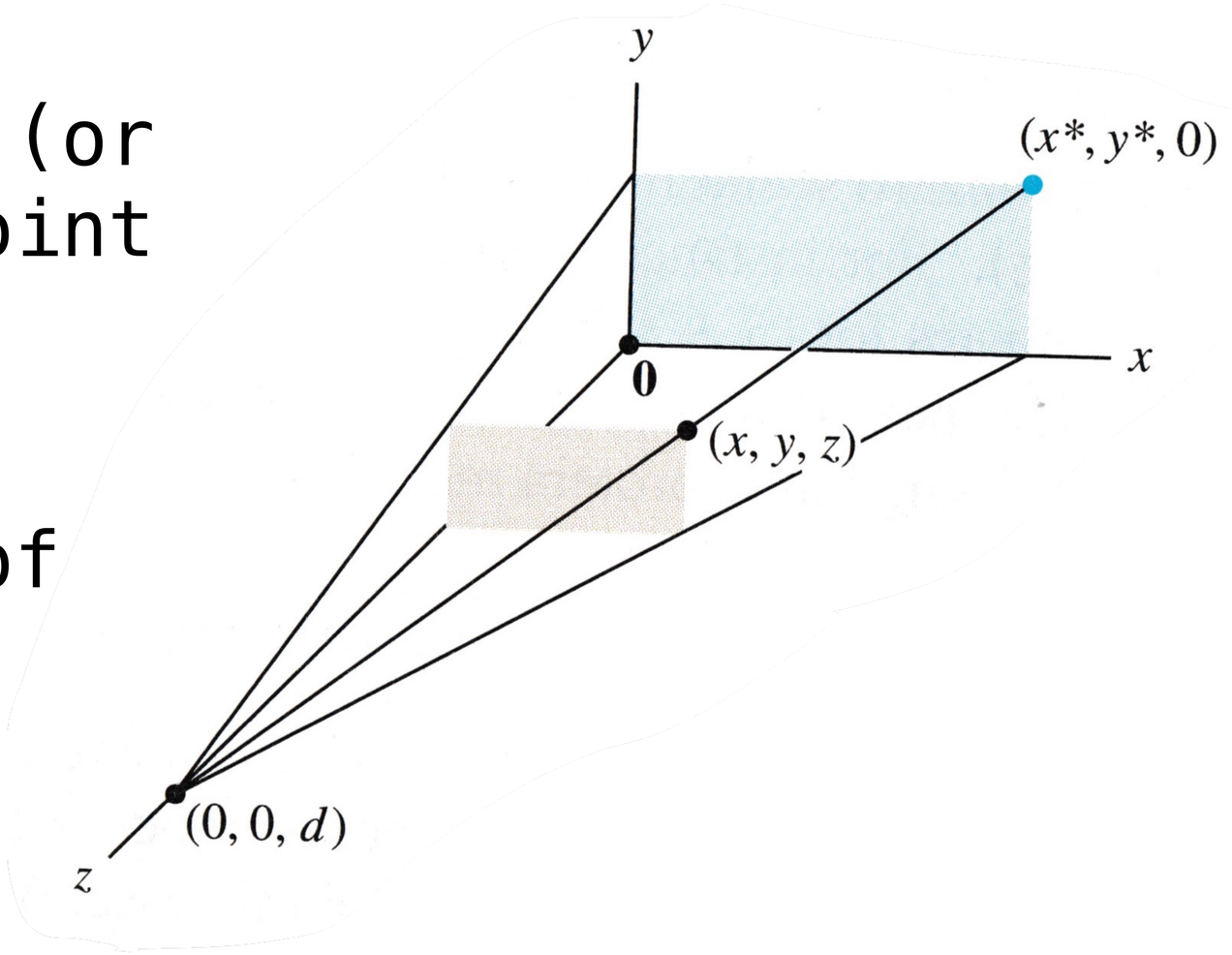
The School of Athens (~1510)



Computing Perspective

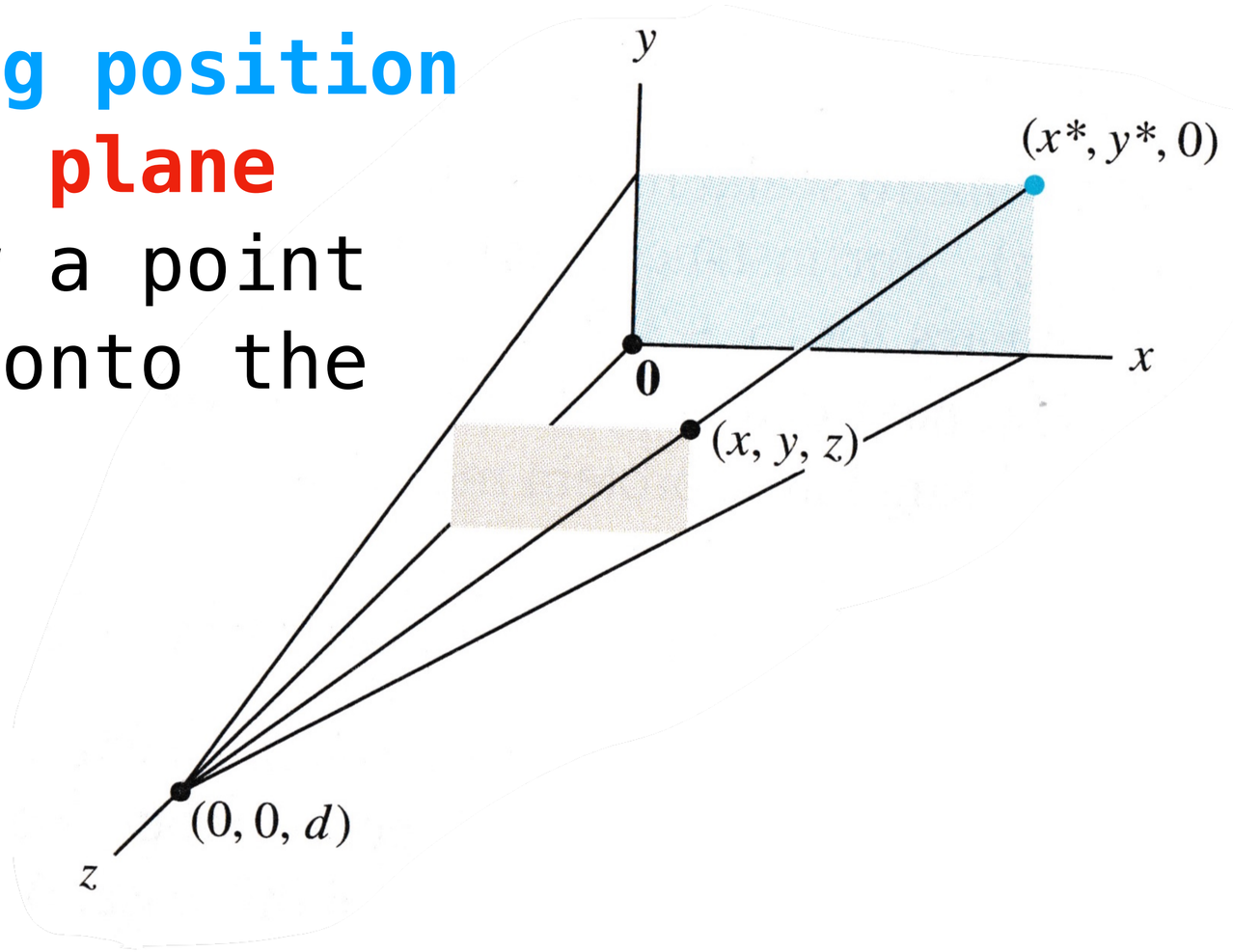
Light enters our eyes (or camera) at a single point from all directions

Closer things "appear bigger" in our field of vision



Computing Perspective

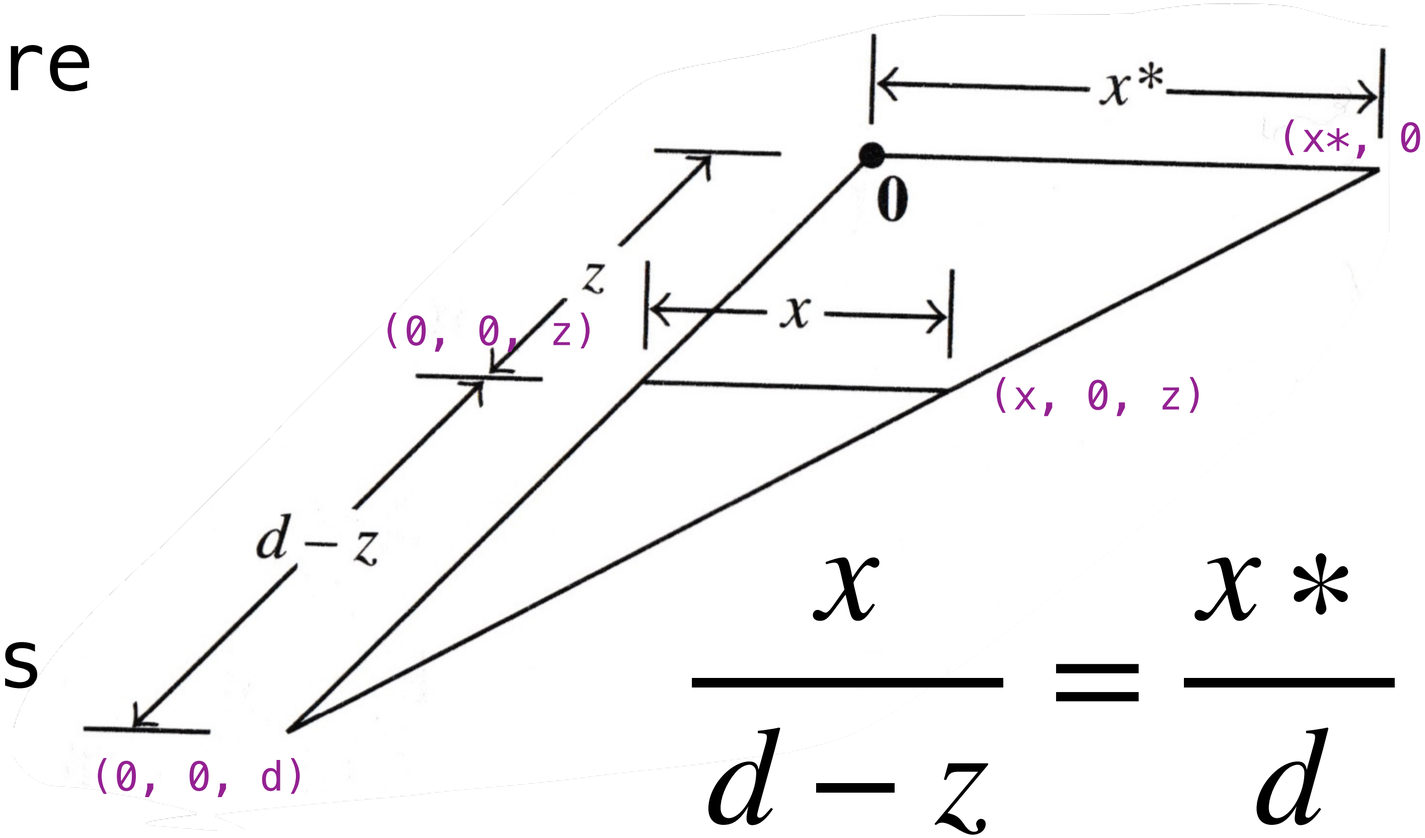
Problem. Given a **viewing position** (0, 0, d) and a **viewing plane** (xy-axis) determine how a point (x, y, z) is *projected* onto the viewing plane



Similar Triangles

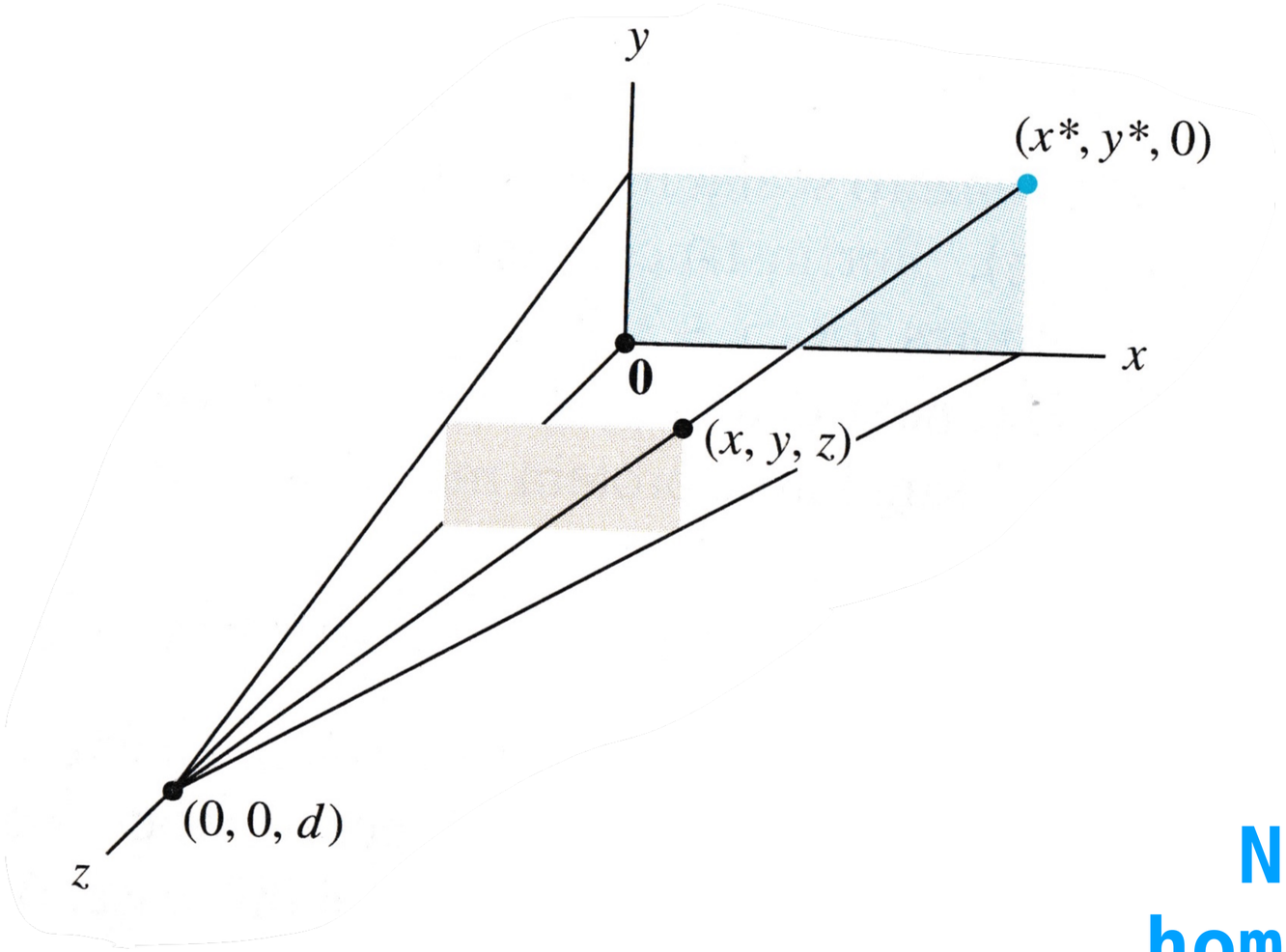
Similar triangles are triangles with the same angles (in the same order)

Similar triangles preserve side ratios



$$\frac{x}{d-z} = \frac{x*}{d}$$

The Transformation



$$x^* = \frac{dx}{d-z} = \frac{x}{1-z/d}$$

$$y^* = \frac{dy}{d-z} = \frac{y}{1-z/d}$$

Not linear, But we use homogeneous coordinates to address this

A Trick with Homogeneous Coordinates

$$\begin{bmatrix} x \\ y \\ z \\ h \end{bmatrix} \mapsto \begin{bmatrix} x/h \\ y/h \\ z/h \end{bmatrix}$$

homogeneous to Cartesian

We can compute perspective using homogeneous coordinates if we allow the extra entry to **vary**

When we convert back to normal coordinates, we divide by the extra entry (this is consistent with before)

Perspective Projection

Definition. The **perspective projection** (and matrix) is given by

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1/d & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ 1 - z/d \end{bmatrix}$$

When we convert back to usual coordinates, we divide by $1 - z/d$ as desired

Lab 4

The Rough Outline

1. Take in a wire frame, represented as a collection of m line segments (pairs of points in $\mathbb{R}^3$)

2. Convert these points into a $4 \times 2m$ matrix D, one column for each endpoint, in homogeneous coordinates

3. Build a transformation matrix A to manipulate the wireframe and project it onto a viewing plane

4. Convert the columns of D into points in $\mathbb{R}^2$, and then pair them back up into endpoints of line segments

5. Draw the resulting image on the screen

demo